从第一步讲起

计算机系统概论

超级纲要 · 教材式初稿

概念 · 推导 · 原图 · 例题 · 自测

先弄懂问题，再看图，再一步一步算。
定义、公式、代码、例题和错误，都放在一起解释。

2026-10-25 · sTBook
Wiki 与 Typst 共用正文 · 原图均在图注中注明出处

章节目录

先把整台计算机串起来 3
1 位、字节、地址：先知道数据放在哪 4
2 整数：同一串位，为什么能有两个答案 6
3 浮点数：它会算小数，但小数格子有空隙 7
4 汇编：把一句 C 拆成机器能执行的小步 9
5 分支和循环：CPU 怎样决定下一步去哪里 10
6 函数调用：走进一个房间，还要记得从哪里回来 12
7 数组、指针和结构体：把地址计算写清楚 14
8 链接：把分散的代码接成能运行的程序 15
9 进程：程序的一次运行，不是一份文件 16
10 信号：一张提醒纸条，不是一份无限排队的消息 17
11 虚拟内存：每个进程看到自己的门牌号 18
12 malloc：拿到的是可用空间，背后还有管理账本 19
13 输入输出：一次 read 不保证读完你要的东西 20
14 线程：同一间办公室里，几个人共用桌面 21
15 同步：什么时候可以动手，什么时候必须等 22
16 性能补充：同样是 O(n)，为什么相差很远 23
17 综合走读：把一段程序从源代码跟到输出 24
18 实战工坊：按位、按地址、按指令走一遍 25
19 系统综合题：把“偶尔错”变成可解释的事件顺序 26
资料覆盖、原图出处与使用说明 27

先把整台计算机串起来

你写下 `printf("hello\n");`，屏幕为什么会出现字？这中间至少发生了这些事：源代码被翻译成指令；链接器找来库函数；操作系统给程序安排地址空间；CPU 取指令并访问内存；库函数把输出送进内核；设备最终显示它。后面的每章，都在回答其中一段发生了什么。

本书先按当前六讲展开整数、浮点、汇编、控制流与函数调用；后半按课程引言指定的 CSAPP 第 7、8、9、10、12 章写完整预习正文。缓存作为理解性能的补充。后续部分不是“老师已经讲过”的记录。原课件仍是课程范围的第一依据。

三个问题，遇到什么代码都问

1. **这些位按什么类型解释？** `11111111` 可以是无符号数 255，也可以是八位补码 −1。内存里的位没有附带一个“我是负数”的标签。
2. **现在处于哪一层？** C 标准、具体 ABI、x86 指令、操作系统各有规则。“我的 CPU 会回绕”不能证明 C 有符号溢出合法。
3. **谁拥有这份状态？** 寄存器属于当前执行上下文；线程共享进程地址空间，但各自有栈。文件描述符和它指向的内核对象也不是同一件东西。

本书的固定环境

主要例子采用 Linux、x86-64、System V AMD64 ABI、AT&T 汇编语法。必要处对比 x86-32。Windows 的 LLP64 数据模型和调用约定不同，不能把 Linux 的 `long` 大小直接照搬。汇编展示的是一种可能的编译结果；优化级别、编译器版本改变后，指令可以变化。

怎么读才不容易累

第一轮读每章开头、图和例子；第二轮拿纸按步骤算；第三轮做自测，再看答案。碰到不懂的缩写，不要硬背：先看它在这一题里具体保存什么、改变什么。新知识很多，不代表每一步都很难。真正容易出错的，往往是把两套规则混在了一起。

1 位、字节、地址：先知道数据放在哪

1.1 一排有编号的小抽屉

一个 bit 只能取 0 或 1；本书的 byte 是 8 bit。内存地址像抽屉编号，每个地址标识一个字节。四字节整数占四个连续地址；变量的地址通常指向其中第一个字节。地址和里面存的数要分开：地址 `0x1000` 的抽屉可以存 `0x7f`。

十六进制每一位刚好对应四个二进制位，因为 $16 = 2^4$。因此 `0x3A` 就是 `0011 1010`，数值为 $3 \times 16 + 10 = 58$。`0x` 是写法标记，不占数据位。1 KiB 是 1024 B，1 kB 是 1000 B；把两者都写成“KB”会留下歧义。

1.2 小端到底小在哪里

以四字节数 `0x12345678` 为例，高位字节是 `12`，低位字节是 `78`。

地址	0x1000	0x1001	0x1002	0x1003
小端存储	78	56	34	12
大端存储	12	34	56	78

小端把低权重字节放在低地址。没有把一个字节内部的 bit 顺序倒过来。网络协议通常规定大端字段，但具体协议仍须看规范；文本字符串按字符编码存储，不能对整串文字套“整数端序”。

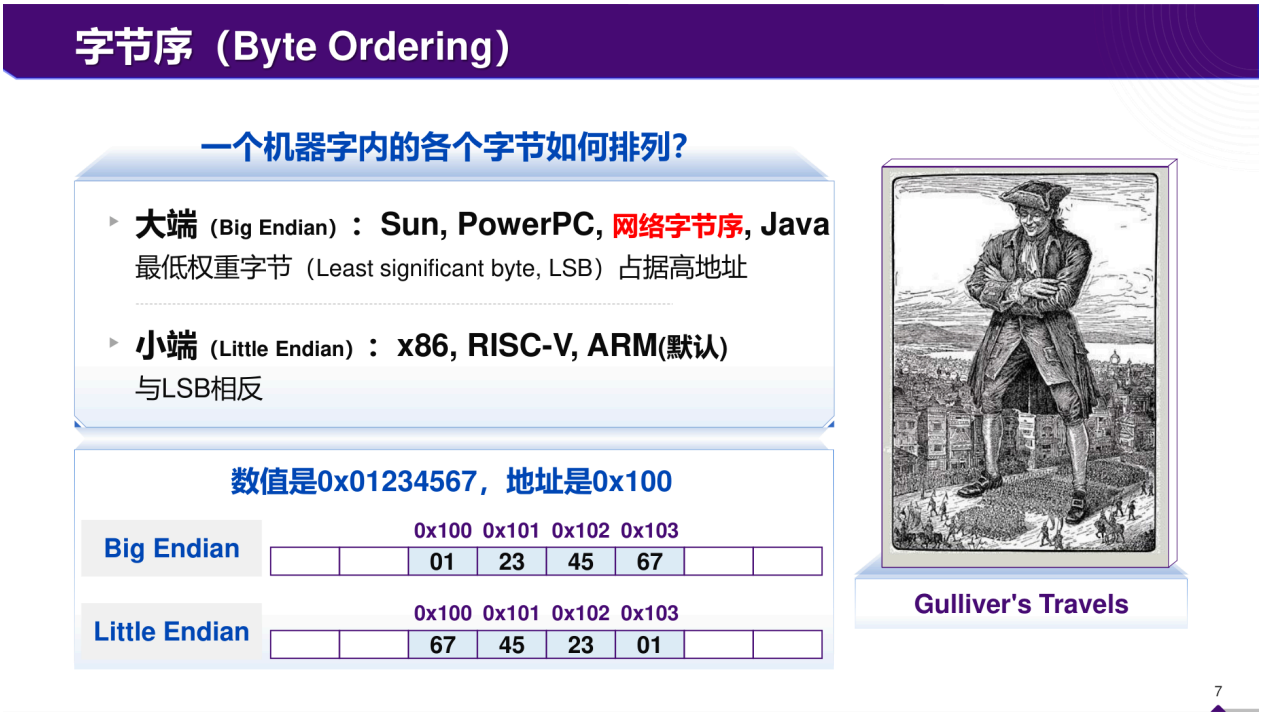


图 1　老师课件原页：数据表示的起点；按下方步骤观察地址和字节。

原图来自《2.1-整数》原始 PDF；准确 PDF 页码见本书“资料覆盖与图源”。

1.3 看一段内存，不靠猜

```c
#include <stdio.h>
#include <stdint.h>
int main(void) {
    uint32_t x = UINT32_C(0x12345678);
    const unsigned char *p = (const unsigned char *)&x;
    for (size_t i = 0; i < sizeof x; ++i)
        printf("%02x ", (unsigned)p[i]);
    putchar('\n');
}
```

先取地址，再把它当字节指针看，最后逐个打印。C 允许用字符类型访问对象的字节表示；这不等于可以随便把任意指针强转成其他类型并解引用，那还会碰到对齐和别名规则。

1.4 数据模型别背成宇宙定律

Linux x86-64 常见 `int=4`、`long=8`、指针 8 字节，称为 LP64；Windows x64 通常 `int=4`、`long=4`、指针 8，称为 LLP64。C 保证的是最小范围和相对约束，具体大小用 `sizeof` 查看。64 位指针也不表示处理器实现了全部 2^{64} 个虚拟地址；不同代处理器支持的有效地址位宽不同。

自测。小端四字节 `01 00 00 80` 是什么无符号数？答：最高权重字节是最后的 `80`，所以是 `0x80000001`，即 $2^{31}+1$。不要从左到右把它直接当十六进制数串读。

2 整数：同一串位，为什么能有两个答案

2.1 先用四个位算一遍

无符号四位数的权重是 8、4、2、1，所以 `1101` 是 $8+4+1=13$。补码把最左位的权重改成 -8，后面不变，所以同一串位是 $-8+4+1=-3$。一般地，w 位补码取值为

$$x = -b_{w-1}2^{w-1} + \sum_{i=0}^{w-2} b_i 2^i.$$

无符号范围是 0 到 2^w-1；补码范围是 -2^{w-1} 到 $2^{w-1}-1$。负数多一个，是因为零占去了非负半区的一个位置。`1000` 是 -8，它的正数 +8 在四位有符号数里放不下。

2.2 为什么“取反加一”能取相反数

四位加法硬件只保留低四位，效果是模 16。3 的位串为 `0011`，逐位取反为 `1100`，加一为 `1101`。两者相加是 `1 0000`，丢掉最高进位后为 0。所以 `1101` 正好充当 3 的加法逆元。注意：对最小补码取反加一会得到它自己；这是位运算事实，不是说数学上 $-(-8)=-8$。

2.3 两种溢出，要看两种标志

四位 `1111+0001=1 0000`：无符号 15+1 超出范围；按有符号解释则是 -1+1=0，并没有有符号溢出。四位 `0111+0001=1000`：无最高进位，但有符号 7+1 超出范围，显示成 -8。无符号看最高进位，有符号看同号相加却得到异号。CF 和 OF 分别服务这两件事。

C 中无符号运算按模 2^w 定义。有符号溢出属于未定义行为：编译器可以假设它不会发生。因此不要用 `x+1<x` 这种有符号表达式检测溢出；应在加法前比较 `x > INT_MAX-y` 等边界，并按符号分情况处理。

2.4 转型：先问值，再问位

把 -1 转成 `unsigned int`，结果是该类型的最大值。混合比较 `-1 < 1u` 时，常见平台会先把 -1 转成 unsigned，比较于是为假。这里失败的不是数学比较，而是参与比较的类型已经变了。

扩宽无符号数通常补零；扩宽有符号补码通常复制符号位。截断只留低位会丢信息。但具体超范围的有符号转换要按所用语言标准和实现判断，别用一条“所有强转只改标签”概括。

2.5 右移不等于所有整数除法

对无符号数右移 k 位等价于除以 2^k 向下取整。C 的有符号整数除法向零截断：$-7/2=-3$。算术右移 -7 常给 -4，二者差 1。若平台采用算术右移，对负数加偏置 2^k-1 后再移位可实现向零截断，但移位宽度必须合法，且要避免中间溢出。以 -7 为例，$(-7+1)\gg 1=-3$。

易错点。 `1 << 31` 不是在所有 C 环境下都安全；表达式先按 `int` 算，最后赋给 64 位变量也救不了前面的溢出。要构造无符号掩码，应从足够宽的无符号 1 开始，并保证移位量小于位宽。

自测。 八位 `10000000` 作为补码和无符号分别多少？答：-128 与 128。为什么不能用八位补码保存 128？因为最大正数只有 127。

3 浮点数：它会算小数，但小数格子有空隙

3.1 二进制小数也有“除不尽”

十进制中 $1/3$ 写不完；二进制中 $1/10$ 写成 $0.000110011\ldots_2$，也写不完。有限二进制小数只能精确表示约分后分母为 2 的幂的有理数。存 `0.1` 时已经舍入，后续运算还可能继续舍入，所以“加十次 0.1 应该严格等于 1”不能当一般浮点测试。

3.2 IEEE binary32 三段分别做什么

binary32 共 32 位：符号 1 位、指数域 8 位、小数域 23 位。对规格化数，设指数域整数为 e、小数域整数为 f，则

$$x = (-1)^s(1 + f/2^{23})2^{e-127},\quad 1 \le e \le 254.$$

前面的 1 不存，叫隐含首位。因此有效精度是 24 个二进制位，不是 23 位。binary64 的域宽为 1、11、52，偏置 1023，有效精度 53 位。

例：$5.75 = 101.11_2 = 1.0111_2 \times 2^2$。符号为 0；指数域是 $2 + 127 = 129 = 10000001_2$；小数域开头为 `0111`，后面补零。拼起来得到 `0x40B80000`。



图 2　老师原页：非规格化数用来填补零附近的空隙。

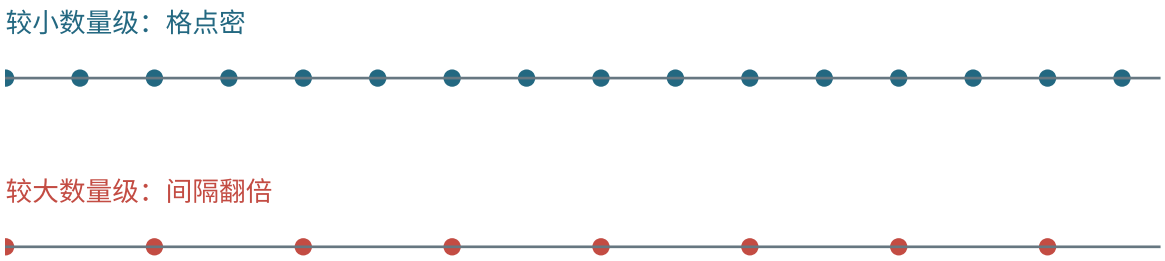
3.3 指数域全零、全一时不能照套

指数域	小数域	含义
全零	全零	正零或负零
全零	非零	非规格化数，$x = (-1)^s(f/2^{23})2^{-126}$
全一	全零	正无穷或负无穷
全一	非零	NaN，不是普通实数

最小正规格化数是 2^{-126}，最小正非规格化数是 2^{-149}。非规格化数没有隐含 1，指数也固定为 −126，从而逐渐接近零。不能把 $e = 0$ 代入 $e - 127$ 再额外补一个 1。

3.4 舍入到最近，正中间选“偶数”

默认常见模式是最近舍入，遇到正中间时选保留结果最低有效位为 0 的那个。以十进制保留整数的类比：2.5→2，3.5→4；真正机器按二进制保留位做，不是按十进制小数位做。这减少了“总往上取”的统计偏向。binary32 在 2^{24} 附近相邻数间隔已为 2，所以 $2^{24}+1$ 舍入回 2^{24}。因此浮点数不满足普通实数的结合律：`(a+b)+c` 和 `a+(b+c)` 可以不同。数量级悬殊时小数可能被吞掉，两个接近数相减又会放大相对误差。



示意相邻可表示数的间隔；真实浮点间隔取决于指数，非均匀分布。

图 3　本书原创图：指数越大，相邻可表示值往往隔得越远。

3.5 怎么比较与转换

按问题的误差预算使用 `abs(a-b) <= atol + rtol*max(abs(a),abs(b))`；接近零需要绝对误差，远离零相对误差更合适。NaN 与任何数比较相等都为假，连它自身也如此，检查应使用 `isnan`。浮点转整数向零截断，但 NaN、无穷或超出目标整数范围不能当正常转换处理。

自测。 `float` 能精确表示从 0 到 2^{24} 的每一个整数吗？答：能；但不能精确表示其后的每一个整数。精度和可表示的最大数量级是两回事：它能放很大的数，却可能相邻差很大。

4 汇编：把一句 C 拆成机器能执行的小步

4.1 寄存器是 CPU 手边的小格子

`%rax` 是 64 位寄存器名，`%eax` 是它的低 32 位，`%ax` 是低 16 位，`%al` 是低 8 位。在 x86-64 中，写 `%eax` 会把 `%rax` 的高 32 位清零；写 `%ax`、`%al` 不会。这一条很容易在逐指令模拟时漏掉。

本书采用 AT&T 语法：**源在前，目的在后**。`movq %rdi,%rax` 是把 rdi 的值复制给 rax，不是反过来。后缀 `b/w/l/q` 分别对应 1/2/4/8 字节。

4.2 立即数、寄存器、内存不是一种东西

写法	读法
`$8`	数字 8 本身
`%rax`	rax 里存的值
`(%rax)`	把 rax 的值当地址，访问那个地址的内存
`8(%rdi,%rsi,4)`	地址为 $8 + R_{rdi} + 4R_{rsi}$ 的内存

例如 rdi=1000，rsi=3，则 `8(%rdi,%rsi,4)` 的有效地址为 1020。缩放因子只允许硬件支持的 1、2、4、8。有效地址不是读出的数据；内存 1020 里可能存着 99。

4.3 `lea` 为何看起来像读内存，却没有读

`leaq 8(%rdi,%rsi,4),%rax` 只计算地址表达式，把 1020 放进 rax；`movq 8(%rdi,%rsi,4),%rax` 则真的访问内存，把那里的八字节读出来。`lea` 还能实现 $3x$、$5x + 7$ 等算术，且不修改条件码。

4.4 逐步走一段

```
# long f(long x, long y) { return x + 4*y + 8; }
leaq 8(%rdi,%rsi,4), %rax
ret
```

按这里的 ABI，x、y 分别从 rdi、rsi 进入，返回值放 rax。x=2、y=3 时，结果 $2 + 12 + 8 = 22$。没有内存访问，也没有真正的乘法指令。不要因为出现括号就判断“这里一定解引用”。

符号扩展与零扩展也不同：把八位 `0x80` 符号扩展会得到 −128 的宽位表示，零扩展则得到 128。读反汇编时不仅看寄存器名字，还要看 `movs*`、`movz*` 等指令的扩展方式。

自测。 rax=4096，内存地址 4096 中存着 7。`movq %rax,%rbx` 和 `movq (%rax),%rbx` 结果各是什么？答：4096 和 7。用“地址的值”和“地址处的值”这两个完整说法，可以减少混淆。

5 分支和循环：CPU 怎样决定下一步去哪里

5.1 先比较，再跳转

`cmpq b,a` 在 AT&T 语法中相当于计算 $a-b$，但不保存差，只更新标志。ZF 表示结果为零，SF 表示结果最高位，CF 反映无符号借位，OF 表示有符号溢出。`testq a,a` 相当于按位与自身，常用于检查零或符号，不改 a。

关系	有符号跳转	无符号跳转
小于	`jl`：SF≠OF	`jb`：CF=1
小于等于	`jle`：ZF=1 或 SF≠OF	`jbe`：CF=1 或 ZF=1
大于等于	`jge`	`jae`
等于	`je`：ZF=1	同样 `je`

为什么有符号小于不能只看 SF？因为做减法时可能溢出。用 SF 与 OF 的组合，才能把这种情况纠正回来。

5.2 同一组位，可以走向不同分支

假设 a 的位全为 1，b=1。按有符号解释，a=-1，所以 a<b；按无符号解释，a 是最大值，所以 a>b。CPU 并没有给寄存器加类型标签，编译器选 `jl` 还是 `jb` 才把 C 类型的意义落实下来。

5.3 while 循环，画三块就够

```
long sum(long n) {
    long s = 0;
    for (long i = 0; i < n; ++i) s += i;
    return s;
}
```

把它拆成“初始化”“判断 i<n”“执行 s+=i、i++、跳回判断”。n=4 时记录状态：开始 (i,s)=(0,0)；四轮后依次是 (1,0)、(2,1)、(3,3)、(4,6)，然后退出。循环不变式是“每轮开始时，s 已经等于 0 到 i-1 的和”。这句话既能解释结果，也能帮助你检查汇编跳转位置。

`do-while` 至少执行一次；`while` 可以一次不执行。优化后的编译器可能把 while 改成先跳测试、或先判一次再采用 do-while 形状。判断源代码不能只数标签，要沿控制流实际走。

5.4 条件移动与 switch

`cmov` 根据标志选择是否复制数据，减少某些分支预测失败。但两侧表达式可能需要先计算，所以不能随意用它实现本来应短路的危险解引用。C 中 `p && *p` 只有 p 非空才访问；强行先算 `*p` 会改变行为。

switch 可编译为比较链、树或跳转表。跳转表适合较密集整数分支，通常先检查索引范围，再按索引取目标地址。出现 `jmp *...` 不意味着函数调用，因为没有自动保存返回地址。

自测。 用 signed 跳转替代 unsigned 跳转，只有遇到负数才错吗？答：从位模式看，只要最高位为 1，就可能改变大小顺序；对无符号数而言那些全是合法的大正数。

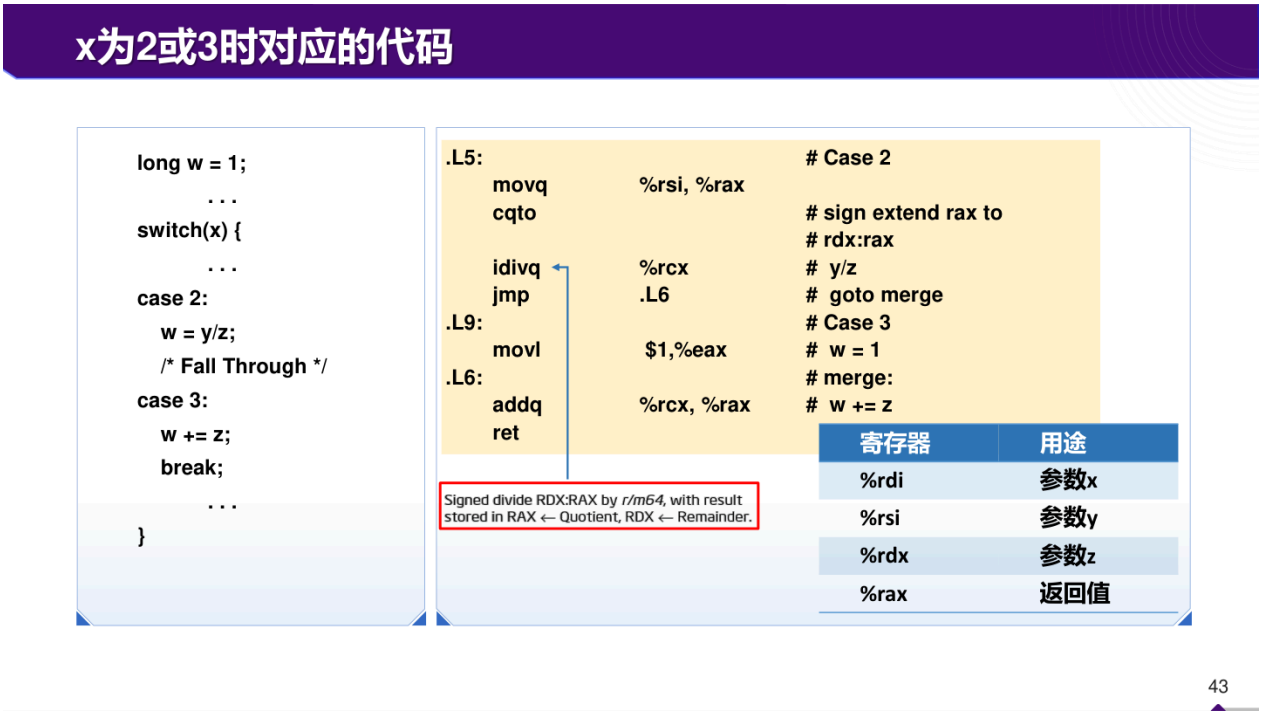


图 4　教师原图：switch 中的贯穿执行。

case 2 算完 y/z 后跳到合流点，再执行 case 3 那段加 z；没有 break 就会贯穿。留意 idiv 使用 rcx 保存除数，因为 cqto 会重写 rdx。参数刚进函数时 z 在 rdx，并不代表后续始终放在那里；要结合前面的寄存器搬运读这一页。

6 函数调用：走进一个房间，还要记得从哪里回来

6.1 一次调用必须交代四件事

参数放哪、返回值放哪、返回地址放哪、哪些寄存器必须替别人保管。这套约定叫调用约定。System V AMD64 中，前六个普通整数/指针参数依次用 rdi、rsi、rdx、rcx、r8、r9；浮点参数有另一套 SSE 寄存器分类，复杂聚合参数还要按 ABI 分类，不能把所有类型都塞进这六个格子。

`call` 把下一条指令地址压栈，再转到被调函数；`ret` 从栈中弹出返回地址并转回。普通整数返回值通常在 rax。栈在常见 x86-64 系统上朝低地址增长；`pushq` 先令 rsp 减 8，再写入数据。

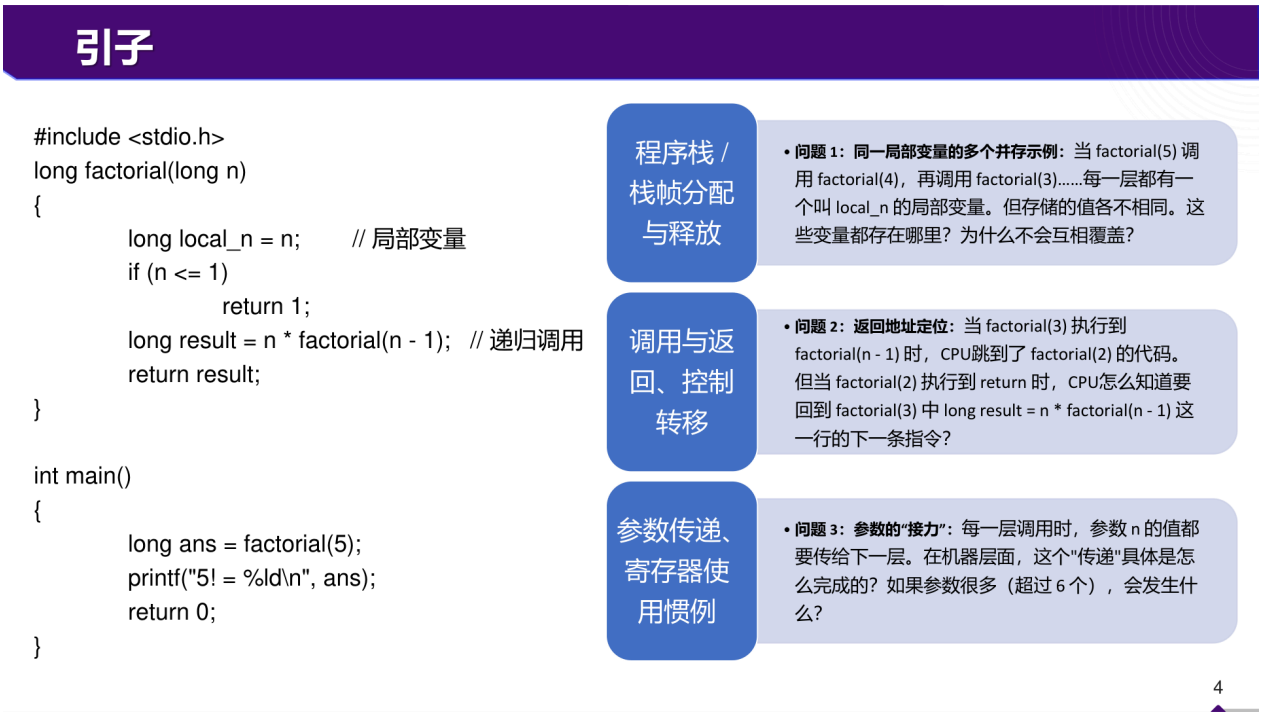


图 5　老师原页：递归需要保存每一层自己的状态。

6.2 谁负责保存寄存器

被调用者保存的常见寄存器包括 rbx、rbp、r12—r15：如果函数改动了它们，返回前要恢复。调用者保存的寄存器可能被被调函数改写，调用者若还要旧值，必须自行保存。rsp 必须恢复到正确位置。rbp 可以作为帧指针，但优化代码可能省略它；没有 `push rbp` 不表示"这个函数没有栈"。

在本 ABI 下，正常函数入口时 rsp+8 应按 16 字节对齐；函数安排局部空间及再次调用前需要维护这一规则。Windows x64 的参数寄存器、shadow space 等另有规定，不要混用。

6.3 手算阶乘，不跳过返回过程

`fact(3)` 调 `fact(2)`，再调 `fact(1)`。下行时保存的是"回来以后还要乘的 n"和返回地址。最深层返回 1；上一层算 2×1=2；最外层算 3×2=6。每一层的 n 是不同存储实例，名字相同不代表共享同一个变量。

递归占用空间看同时活着的调用深度。普通阶乘深度 n，辅助栈空间 $O(n)$；二分搜索深度 $O(\log n)$。所有调用次数的总和是时间分析，不是峰值栈空间。

6.4 x86–32 对照

传统 32 位 cdecl 通常把参数从右往左压栈，返回值放 eax，调用者清理参数；也存在其他约定。x86-64 的寄存器传参不能直接移植到 32 位反汇编题。先看寄存器宽度和课题设定，再解释栈偏移。

6.5 栈越界为什么危险

局部数组和保存的控制信息可能相邻；越界写会破坏别人的数据，甚至返回地址。安全写法要检查长度，使用有界接口，不把“运行一次没崩”当正确证明。本书只用原理解释课程实验，不接触真实系统攻击目标。

自测。 递归返回时，rsp 每层少恢复 8 字节，会发生什么？答：后续 `ret` 会把错误位置的数据当返回地址，控制流和栈状态都可能损坏。



图 6　教师原图：栈帧、栈顶与帧指针。

图上较高地址是先前调用的状态；rsp 标当前栈顶，rbp 若使用则充当本帧稳定参考点。某函数可以没有 rbp 帧指针，也可以把部分局部变量留在寄存器里，所以不要把示意图当每段优化代码都必须逐格照搬的固定模板。

7 数组、指针和结构体：把地址计算写清楚

7.1 数组访问本质上是“起点加偏移”

`int a[5]` 的起点是 B，且本平台 `sizeof(int)=4`，则 `a[i]` 地址为 $B+4i$。二维连续数组 `int a[R][C]` 按行排列，`a[i][j]` 的地址为 $B+4(iC+j)$。行宽 C 决定跨行步长；不是 $B+4(i+j)$。

`int **p` 则通常先读取一根指针，再沿那根指针访问对应行，各行不必连续。它与二维数组不是同一种布局，不能只因为都写两个下标就互相强转。

7.2 结构体为什么会长出空白

```
struct Example { char tag; int count; short flag; };
```

在常见 x86-64 ABI 中，tag 在偏移 0，count 要按 4 字节对齐，放偏移 4，flag 放偏移 8。结尾再补齐到结构体对齐要求的整数倍，因此总大小通常 12，而不是 1+4+2=7。具体仍以 ABI 和 `offsetof` 为准。

补齐空间不是可以依赖的业务数据。直接比较两个结构体的全部字节可能被填充字节干扰；直接把结构体内存发到网络还会遇到端序、填充、大小和隐私残留问题，应显式编码字段。

7.3 几种“看起来还活着”的坏指针

- 返回局部数组地址：函数结束后对象生命期已结束，地址数字还在也不能再用。
- `free(p)` 后继续读 p：释放不一定清零，读到旧数据也属于失效访问。
- 扩容后继续使用旧元素指针：新的存储区可能换地址，旧指针已经悬空。
- 下标等于数组长度：它是尾后位置，只能在规定范围内作指针运算，不能解引用。

7.4 把一段寻址反推成 C

若指令计算 `base + 8*i + 24`，先列可能：八字节数组第 i 项再偏移 24，或结构体数组中的某字段。仅凭一条指令未必唯一；结合访问宽度、循环步长和相邻指令确定。反汇编题追求与机器行为一致的表达，不保证恢复原程序员的名字和写法。

例题。 `double a[3][4]`，B=1000，a[2][1] 在哪？答：每行 4×8=32 B；先跨两行到 1064，再跨一项到 1072。先算行步长，再算列偏移，不容易乱。

自测。 结构体中存在 char 成员，是否保证整个结构体只需按 1 字节对齐？答：不保证；通常要满足所有成员中最大的对齐要求。

8 链接：把分散的代码接成能运行的程序

8.1 一份源文件并不是整个程序

预处理展开头文件和宏；编译把 C 翻译为汇编；汇编生成可重定位目标文件；链接器把多个目标文件和库组合起来。`.o` 里通常还不知道其他文件的函数最终放在哪，所以既保存机器码，也保存“这里以后要填地址”的记录。

符号表回答“这个名字在哪个节、哪个位置”；重定位记录回答“哪处指令或数据需要按符号地址修正”。同一名字的局部变量不一定参与链接；函数内自动局部变量多数存在栈或寄存器中，而文件作用域变量与函数通常需要链接器处理。

8.2 先分清三种 static

文件作用域 `static int x` 限制为本翻译单元内部链接；函数内 `static int x` 具有静态存储期，跨调用保留值，但不是普通栈变量；C++ 类的 static 成员又涉及类语义。它们共享词形，不表示同一个机制。`extern int x;` 通常是声明，不分配一份新的普通定义。

旧教材的“强符号/弱符号”讲法依赖工具链策略。现代 GCC 默认 `-fno-common`，多个翻译单元重复写 `int x;` 可能直接报多重定义；不能把旧版 common 合并规则当 C 语言允许重复定义的保证。可靠做法是一个地方定义，其余头文件声明。

8.3 一个最小重定位例子

文件 a.c 调用 `f()`，b.c 定义 f。编译 a.c 时先留占位，链接后才知道 f 的相对位置。若调用指令采用 PC 相对位移，逻辑上需要计算

$$\text{位移} = \text{目标地址} - \text{下一条指令地址}\ .$$

具体 ELF 重定位项还涉及 S、A、P 的定义，不能把“下一条地址”与重定位字段地址混为一谈。解题时先写清楚字段位置、指令长度和所用重定位类型，再代数。

8.4 静态库的顺序为什么重要

传统静态库按命令行从左往右扫描，为当前未解决的引用提取成员。因此常写 `main.o -lfoo`，先产生对 foo 的需求，再去库里找。循环依赖可能需要分组扫描或调整结构。库里没被引用的对象不一定全塞进最终文件。

动态库让代码在装载或运行时共享。GOT 保存间接地址，PLT 常用作函数调用跳板；惰性绑定是在第一次调用时解析，但系统也可以配置立即绑定。PIC 通过位置无关的访问方式支持装载到不同地址，不能简单理解为“程序里再也没有地址”。

自测。 两个源文件都包含同一个头文件，头文件中写了普通全局变量定义，为什么可能失败？答：包含是文本展开，两边各产生定义；应把定义放进一个源文件，在头文件里写 extern 声明。

9 进程：程序的一次运行，不是一份文件

9.1 异常不只有“出错”

CPU 正常沿指令流执行，但外部中断、系统调用、缺页和非法指令都可能让控制转入内核。中断通常异步出现；陷阱常用于有意请求服务；故障有些可以修复后重试，比如合法页面尚未驻留；不可恢复的严重情况则终止。课程的分类与具体体系结构术语要对照，不要把所有异常都叫崩溃。

进程提供两种重要感觉：好像独占 CPU，好像独占内存。实际由调度切换和虚拟内存实现。程序是磁盘上的代码与数据；进程还包括运行中的寄存器、地址空间、文件描述符等状态。

9.2 fork 为什么“一次调用，两个返回”

调用前只有父进程；成功 fork 后父子都从调用后的语句继续。父进程得到子 PID，子进程得到 0；失败只在父进程返回 −1，没有子进程。子进程看起来得到独立地址空间，常通过写时复制实现；之后普通局部变量各自修改，不会自动互相通信。

```
int x = 10;
pid_t pid = fork();
if (pid == 0) { ++x; printf("child %d\n", x); }
else if (pid > 0) { printf("parent %d\n", x); }
```

成功时分别输出 11 和 10，但先后顺序不固定。不要因为代码先写 child 分支就断定子先运行。标准输出缓冲区也会被复制：fork 前未刷新的数据可能被双方再次刷新，造成看似多打印一次。

9.3 exec、exit、wait 各做一件事

exec 系列用新程序替换当前进程映像，成功时不返回；不是再造一个进程，因此 PID 通常保持。exit 结束进程并执行库层清理，_exit 跳过用户态 stdio 等清理；两者使用场合不同。父进程用 wait/waitpid 回收子进程退出状态；只退出但未回收的子进程可能处于僵尸状态。

9.4 画树再数输出

连续两次 fork 且全部成功、每个进程都执行两次时，数量从 1→2→4。若某个分支提前 exit，或只让父进程第二次 fork，就不能套 2^n。逐次画出“谁会执行下一行”，比凭经验猜更稳。

自测。exec 成功后会执行它后面的 `printf("done")` 吗？答：不会；当前程序已被替换。只有 exec 失败返回，才可能走到后面的错误处理。

10 信号：一张提醒纸条，不是一份无限排队的消息

10.1 先区分“产生、待处理、递送”

某事件产生信号；若暂时阻塞，它会处于待处理状态；允许递送时，内核安排默认动作或信号处理函数。传统标准信号通常不按次数逐条排队：同一种信号连续产生多次，可能只留下一个待处理位。实时信号另有排队语义。

因此 SIGCHLD 到来后，不能只调用一次 waitpid 就认为全部子进程已回收。应在允许的上下文与设计下循环 `waitpid(-1, &status, WNOHANG)`，直到没有可立即回收的子进程，并处理错误和 EINTR。

10.2 主程序和处理函数会互相打断

假设主程序正在修改链表，信号处理函数又来读取它，可能恰好看到“只改了一半”的状态。处理函数也不能任意调用 printf、malloc 等非异步信号安全函数；被打断的代码可能正持有它们内部的锁。一个简单模式是处理函数只设置 `volatile sig_atomic_t` 标志，主循环在合适位置处理工作。它适用于受限的信号通知，不等于通用线程同步原语。

10.3 那个很小的时间窗口为什么重要

典型坏例子：父进程 fork 子进程，准备把 PID 加入任务表；子进程非常快地退出，SIGCHLD 处理先来删除；父进程随后才加入，于是留下永远不会再被删除的记录。问题不在 CPU 太慢，而在操作顺序没有约束。

一种办法：fork 前阻塞 SIGCHLD；父进程添加记录后恢复掩码；子进程也恢复继承的掩码后再 exec。这样通知不会在任务表尚未准备好时被递送。注意 fork 会继承信号掩码，因此漏掉子分支恢复会给新程序留下意外环境。

10.4 等事件，不要“看一眼再睡”

先检查标志，再调用 pause，中间可能恰好收到信号：信号已处理完，程序却随后睡下，可能再也等不到。`sigsuspend` 能原子地临时更换掩码并等待信号，用来关闭这个窗口。必须配合循环重新检查条件，因为唤醒不保证目标条件已成立。

自测。收到一次 SIGCHLD，能否推断只有一个子进程结束？答：不能；标准信号可能合并，必须查询真实子进程状态。把信号当“去看看”的提醒，通常比当精确计数器安全。

11 虚拟内存：每个进程看到自己的门牌号

11.1 地址先翻译，再访问物理内存

程序产生虚拟地址。MMU 根据当前进程页表翻译成物理地址，页表项还带有效、权限等信息。两个进程都访问虚拟地址 0x400000，可以映射到不同物理页；共享内存也可以让不同虚拟地址映射到同一物理页。若页大小为 2^p 字节，地址低 p 位是页内偏移，高位是虚拟页号。翻译改变页号，不改变页内偏移。以 4 KiB 页为例，p=12，虚拟地址 `0x12345` 的页号为 `0x12`、偏移为 `0x345`。若页表映射到物理页框 `0x9a`，物理地址是 `0x9a345`。

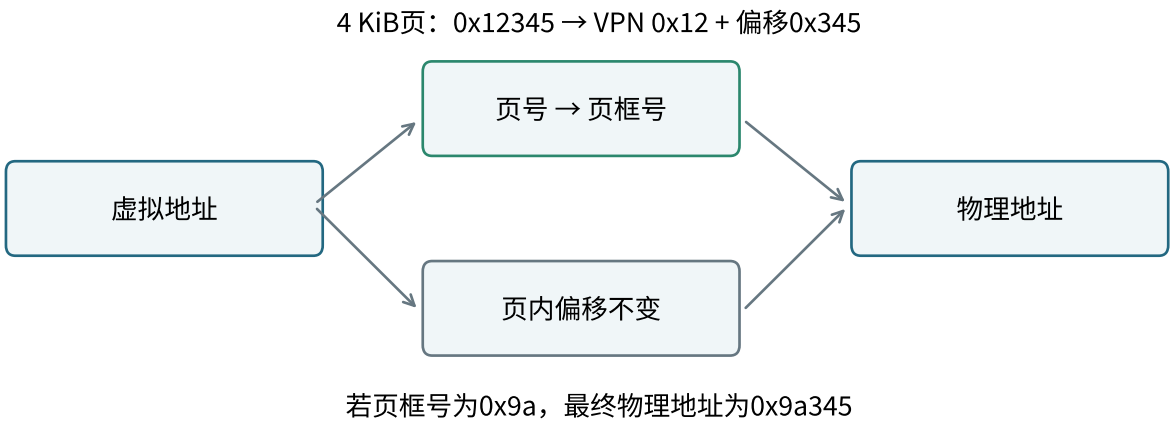


图 7　本书原创图：页号经过翻译，低位偏移直接保留。

11.2 TLB 只是翻译的缓存

TLB 缓存近期页表映射；命中时不必再次遍历页表。TLB 未命中不等于缺页：映射可能有效，只是 TLB 里没有。缺页又不一定非法：页面可能已分配但暂未驻留，内核把它调入后重试指令。若根本没有合法映射或权限不符，则是另一种处理结果。

课程引言注明不要求多级页表，本书仍解释它为何节省稀疏地址空间的页表存储，但不把复杂多级位拆分当默认考核任务。具体层数、页大小和地址位宽取决于体系结构及配置。

11.3 四种访问，别混成“内存都差不多”

1. 合法且已驻留：翻译后访问。
2. 合法但未驻留：缺页，调入后重试。
3. 只读页写入：可能是非法写，也可能触发写时复制。
4. 根本没有合法映射：不能靠调入页面修复普通越界。

写时复制让 fork 后父子先共享只读物理页；某方尝试写入时，再给它复制独立页面。这解释了“逻辑上各有一份内存”和“物理上暂时共享页面”为何能同时成立。

11.4 mmap、共享和文件

mmap 把文件或匿名区域映射进地址空间。私有映射的修改通常不写回原文件；共享映射允许共享修改，但一致性和持久化仍有具体语义。虚拟地址连续不代表物理内存连续，反过来也一样。

自测。 4 KiB 页中，地址 0x1fff 与 0x2000 是否同页？答：不是；前者页号 1、偏移 0xfff，后者页号 2、偏移 0。跨页可能需要第二次翻译，并可能遇到不同权限。

12 malloc：拿到的是可用空间，背后还有管理账本

12.1 申请 13 字节，分配器可能占得更多

分配器要满足对齐、保存块大小和状态，必要时还要保留空闲链表指针。用户只拥有返回地址开始的所请求字节，不能因为“实际块好像更大”就越过请求大小写入。

内部碎片是已分配块内的浪费，例如对齐补齐；外部碎片是空闲总量不少，却被分成互不相邻的小块，满足不了一大块连续请求。两者的改进方向不同。

12.2 一块内存从哪里找

隐式空闲链表按块大小遍历所有块；显式空闲链表只链接空闲块；分离适配按大小分组。first fit 找到第一块足够大的就用；best fit 找最接近需求的块，可能减少当前剩余，但查找更贵，也不保证长期最少碎片。

假设空闲块为 32、80、48 B，要 40 B：first fit 选 80，best fit 选 48。若剩余部分小于最小合法块，就不应硬拆，否则会制造无法管理的小碎片。

12.3 分裂、合并和边界标记

拿到大块后可分裂为已分配块和空闲尾块。释放时，检查前后物理相邻块是否空闲，合并为更大块。注意“链表相邻”和“物理相邻”不同：链表里的上一项可能在完全不同地址。边界标记常用头部/尾部保存大小，让分配器能找到前一个物理块；具体优化方案可能省略已分配块的尾部。

双重释放可能把同一块重复加入空闲结构，破坏元数据；释放后继续写会改坏分配器账本。`free(NULL)` 在 C 中允许，但 free 一个并非有效分配起点的地址不允许。

12.4 realloc 不保证原地不动

成功时可能移动到新地址并释放旧块；失败返回 NULL 时，原块仍有效。因此不能直接 `p = realloc(p,n)` 后忽略失败，否则可能丢失原指针。用临时指针承接结果，再在成功后更新 p。零大小请求还有语言版本和实现边界，应避免用它表达含糊的业务意图。

自测。 总共空闲 1 MiB，是否保证能申请 1 MiB 的连续块？答：不保证；可能分散在许多小块中，还要考虑元数据、对齐和其他限制。

13 输入输出：一次 read 不保证读完你要的东西

13.1 文件描述符是一个小整数句柄

open 返回文件描述符，read/write 用它请求内核操作。进程的描述符表指向内核的打开文件状态，再指向文件等对象。fork 后父子对应描述符可能共享打开文件描述的偏移；dup 也会让两个描述符指向同一个打开文件描述。两个独立 open 则通常有各自偏移。

因此“两个 fd 数字不同”不保证读写位置独立；“两个进程”也不保证偏移独立。反过来，同一个 fd 数字在不同进程中完全可以指向不同对象。

13.2 短读、短写是正常情况

`read(fd, buf, n)` 返回实际读取字节数，可能小于 n。返回 0 表示该次读取遇到 EOF；返回 −1 表示错误，要检查 errno。管道、终端、网络和信号中断都可能让一次调用无法完成整个逻辑消息。

可靠读写循环应维护“已完成量”和“剩余量”，每成功一次就移动指针；对 EINTR 按接口语义处理；遇到 EOF 时把“完整消息”“提前结束”区分开。不能因为暂时只来了 20 字节，就当后面的 80 字节永远不存在。

```
done = 0
while done < need:
    n = read(fd, buffer + done, need - done)
    if n > 0: done += n
    else if n == 0: return 提前EOF(done)
    else if errno == EINTR: continue
    else: return 错误
return 完成(done)
```

这里的 read 是阻塞式简化模型。非阻塞 fd 遇到 EAGAIN 需要等待就绪等机制，不能不加限制地死循环重试。

13.3 stdio 缓冲和内核不是一层

printf/fread 等库接口可以先把数据留在用户态缓冲区，减少系统调用次数；write 直接进入内核接口。混用时要留意刷新顺序，fork 时也要留意复制缓冲。写进内核缓存不一定已经落到持久介质，fflush 与 fsync 的职责不同。

重定向的核心是让标准输入/输出描述符指向其他打开对象，常用 dup2。打开标志、文件元数据查询和目录操作也都围绕“句柄指向什么对象、对象具有什么状态”来理解。

自测。 read 返回 7，而你请求 100，是否是报错？答：不是；7 是成功读到的数量。只有返回 −1 才进入错误检查。

14 线程：同一间办公室里，几个人共用桌面

14.1 共享什么，各自有什么

同一进程的线程通常共享全局变量、堆和地址空间，各自有寄存器、执行栈、线程 ID 等。某线程的局部变量“通常在自己的栈上”，不代表其他线程绝对访问不到：若把它的地址传出去，其他线程仍可能通过指针读写，直到生命期结束。

创建线程时要给参数稳定的存储。循环里反复传 `&i`，所有线程可能看见同一个正在改变的 `i`；应给每个线程各自的参数对象，并保证使用期间它们仍存活。在线程退出前释放参数、或把已离开作用域的栈地址留给线程，都是另一种生命期错误。

14.2 `count++` 为什么会丢更新

把它拆成读、加一、写回。初值 0：A 读 0，B 也读 0，A 写 1，B 写 1，结果只有 1。即使某平台偶尔把它编成一条内存指令，也不能自动推导为满足语言层线程原子性。

C/C++ 中，对同一非原子对象无同步地发生冲突访问，且至少一方写入，可能构成数据竞争并导致未定义行为。`volatile` 不会把普通变量升级成互斥锁或线程原子变量。

14.3 并发、并行与速度

并发指多个任务在时间上交错推进；并行指真的同时运行。单核也能并发，多核才可能让 CPU 工作并行。并行程序的串行部分会限制加速：若串行比例 s，理想 p 核加速上界为

$$S(p) = \frac{1}{s + (1 - s)/p}.$$

例如 s=0.1，p→∞ 时也只能到 10 倍。这还没算同步、内存带宽等开销，真实速度通常更低。

14.4 可重入和线程安全不是同义词

返回静态缓冲区地址的函数可能被下一次调用覆盖。加锁可以让它线程安全，却不一定适合在信号处理函数中重入。检查函数要看其状态藏在哪里、谁会同时改、接口如何交付结果，不能只看它有没有局部变量。

自测。 两个线程各写不同变量，就一定互不影响性能吗？答：逻辑上可能无数据竞争，但若变量落在同一缓存行，仍可能产生伪共享。正确性和性能需要分别检查。

15 同步：什么时候可以动手，什么时候必须等

15.1 互斥锁守住一段完整动作

锁保护的是不变量，不只是某一条赋值。例如银行余额转账要让“扣款与入账”作为一个受保护的逻辑过程；只给两个单独赋值各加一次锁，中间状态仍可能被看见。临界区要覆盖维护一致性所需的全部访问，同时避免把无关慢操作放进去。

计数信号量的 P 操作等待并扣减可用计数，V 操作增加计数并可能唤醒等待者。它适合表达资源数量；二元信号量可用于互斥，但接口语义与带拥有者约束的 mutex 不完全相同。

15.2 生产者消费者，逐项解释

容量 N 的缓冲区设 `empty=N`、`full=0`、`mutex=1`。生产者先 P(empty) 占一空位，再 P(mutex) 修改缓冲，之后 V(mutex)、V(full)。消费者先 P(full) 确认有物品，再 P(mutex) 取出，之后 V(mutex)、V(empty)。

为什么不要先拿 mutex 再等 empty？缓冲已满时生产者拿着 mutex 睡下，消费者又必须拿 mutex 才能腾空位，双方就互相等。信号量顺序并非可以随手交换。

条件变量适合“某条件尚未满足就睡眠”。等待必须与受同一互斥锁保护的条件检查配合，并用 while 重新检查，而不是 if：醒来时条件可能已被其他线程改变，也可能出现允许的伪唤醒。

15.3 死锁怎么一步一步形成

A 拿 L1 等 L2，B 拿 L2 等 L1；它们各自持有对方需要的资源，又都不释放。常用防法是给锁规定统一顺序：所有线程都先 L1 后 L2。这样不能形成相反的等待环。把“偶尔没复现”当不存在，无法解决竞态或死锁。

读写锁适合多读少写，但读者优先可能让写者饥饿，写者优先也可能影响读者延迟。选择策略必须看工作负载和具体实现，不能只背“读写锁更快”。

自测。 线程 A 持锁后遇到错误直接 return，忘记解锁，最可能怎样？答：其他线程可能永远阻塞。C++ 可用 RAII 锁对象绑定作用域；C 则应设计统一清理路径。

16 性能补充：同样是 O(n)，为什么相差很远

16.1 快的存储小，慢的存储大

寄存器、缓存、主存、外部存储组成层次。缓存按行搬运数据，程序若刚访问一个地址，又访问附近地址，叫空间局部性；很快再访问同一地址，叫时间局部性。行优先连续扫描二维 C 数组通常比逐列跳跃更友好。

设缓存块大小为 2^b、组数为 2^s，地址常分成 tag、组索引、块内偏移。块内偏移取低 b 位；组索引取接下来的 s 位；剩下是 tag。同一组的不同块可能互相替换，因而出现冲突失效。TLB 的缓存对象是页表翻译，数据 cache 的对象是内存数据，两者别混。

16.2 平均访问时间，先写概率

若命中时间 h，失效率 m，额外失效代价 p，则常见简化模型是 AMAT $= h + mp$。例如 h=1 ns，m=0.05，p=60 ns，得到 4 ns。这里 p 是“额外”惩罚，若题目给的是包含命中部分的总未命中时间，公式需要相应调整。

16.3 复杂度没变，常数也不是玄学

链表每个节点可能散在不同位置，指针追踪让下一个地址依赖上一次读取；向量连续存储，硬件更容易预取。向量插入要移动后缀，链表在已知节点附近可常数次改链接。不是谁永远更快，而是看访问方式、元素大小、定位成本和分配开销。

编译优化也受别名影响：如果两个指针可能指向同一处，编译器不能随意把读写重排。`restrict` 等承诺有严格前提，违反承诺会出错。优化前应先测量，避免为了“减少一条指令”制造未定义行为。

例题。 一个 1024×1024 的 double 矩阵每行 8192 B。逐列访问时相邻两次隔 8192 B；逐行相邻只隔 8 B。两者都是访问约一百万项，但缓存行利用率、预取和 TLB 行为可能大不一样。

自测。 AMAT 较小是否保证程序一定更快？答：不保证；总访存次数、计算量、并发和带宽瓶颈等还会共同影响总时间。

17 综合走读：把一段程序从源代码跟到输出

17.1 一次完整的思考顺序

假设程序读入 n 个整数，malloc 一块数组，累加后输出结果。按下面顺序检查，比一上来盯着循环更完整。

1. **输入能不能信？** n 是否成功解析、是否为负、是否超过内存预算？
2. **字节数会不会溢出？** 申请前检查 `n <= SIZE_MAX / sizeof *a`，不要让 `n*sizeof *a` 先回绕成小数。
3. **申请结果有效吗？** malloc 失败必须处理；零大小的业务含义要明确。
4. **累加类型够不够？** 每项是 int，不代表总和能放进 int；用更宽类型也要估计最终界限。
5. **实际读到了多少？** 文件可能提前结束，系统级 read 可能短读；没有完整数据不能继续当真。
6. **谁负责释放？** 正常返回和出错路径都要照顾，释放后不能继续使用旧指针。
7. **若加线程会怎样？** 线程先各算局部和，再用受保护或安全归约方式合并；不让所有线程竞相执行普通 `sum+=a[i]`。

17.2 三道有过程的例题

题一：补码与比较。八位 `a=11111110`、`b=00000001`。a 的 unsigned 值是 254，signed 值是 −2；因此 unsigned 的 a>b，而 signed 的 a<b。位串没变，比较规则变了。

题二：地址翻译。4 KiB 页，虚拟地址 0x2abc，虚拟页 2 映射到物理页框 7。先取偏移 0xabc，再把页框号左移 12 位，得 0x7000，合并为 0x7abc。若写该页而页表只读，应先处理权限，不能直接给出“写成功”。

题三：丢更新。两线程都将 x 从 4 加 1：A 读 4，B 读 4，A 写 5，B 写 5。这个交错展示了错误机制；在 C/C++ 中，若 x 是普通共享变量且无同步，进一步涉及数据竞争和未定义行为，不能把“结果只可能 5 或 6”当完整语言结论。

17.3 用自己的话回答这些问题

- 为什么 32 位整数加法的硬件回绕，不能保证 signed C 加法合法？答：指令层与语言层约定不同，优化器依赖语言规则。
- 为什么 TLB miss 不一定 page fault？答：翻译不在缓存，不代表映射无效或数据不在内存。
- 为什么 fd 不同不保证偏移独立？答：它们可能指向同一个内核打开文件描述。
- 为什么“锁住写、不锁读”仍可能错？答：读者可能与写者冲突，且看见中间状态；同步必须覆盖不变量相关的所有访问。
- 为什么截图上一组汇编不是唯一正确答案？答：编译器可以使用不同等价指令序列，行为与前提才是比较标准。

17.4 一张真正有用的复习纸

写四栏：位宽与范围；指令读写与条件码；进程/线程/内核对象的共享关系；所有公式的适用条件。每栏再放一个亲手算过的小例子。只有公式而没有“什么时候能用”，考题换个前提就容易出错。

18 实战工坊：按位、按地址、按指令走一遍

18.1 题一：从 short 到 unsigned，中间经过什么

假设 short16 位、int32 位，s=-12345。先把绝对值写为 0x3039，16 位取反加一得 0xCFC7。若小端存储，内存依次 C7 CF。整数提升到 int 时符号扩展为 0xFFFFCFC7；再转 unsigned int 按模 2^{32}得到 4294954951。转型前后用的不是同一个取值范围，不能把 16 位的 53191 直接当最终 unsigned int 结果。

检查：2^{32}-12345=4294954951。若直接把同一 16 位位串重新解释成 unsigned short，才得到 2^{16}-12345=53191。转换路径不同，结果不同。

18.2 题二：八位补码加法与标志

0x7f+0x01 低八位为 0x80。有符号 127+1 越界，OF=1；无符号 127+1=128 合法，CF=0。0xff+0x01 低八位 0，CF=1；有符号-1+1=0 合法，OF=0。两题都能由同一个加法电路完成；解释结果依赖你读哪个标志。

18.3 题三：一个教学小浮点数

采用课件式 1 符号+4 指数+3 小数、偏置 7、指数全零/全一按 IEEE 式特殊规则的教学格式。最小正规格化 2^{-6}，最小正非规格化 2^{-9}，最大有限值$(2-2^{-3})\cdot 2^{7}$=240。

7.25=111.01_2=$1.1101_2\times 2^2$。只有 3 个小数位时，它正好在 $1.110_2\times 4$=7 与 $1.111_2\times 4$=7.5 中间。最近偶数舍入选保留末位为 0 的 110，结果 7；指数域 2+7=9，小数 110，位串 0 1001 110。这里“偶”看二进制保留位，不看十进制个位 7 是否奇数。

这是一种教学格式，不能把它的所有特殊值规则不加区分地套给真实硬件中每一种 E4M3/FP8 变体；有些格式采用不同的无穷/NaN 约定。

18.4 题四：一段真实可出现的地址计算

参数 a 起点 0x1000，i=3，`int a[][5]`，访问 a[2][3]。偏移=(2×5+3)×4=52=0x34，地址 0x1034。若是 `int **p`，则需先访问保存第 2 行指针的位置，再沿该指针偏移 12，不能把 0x1034 直接当通用答案。

18.5 题五：手工栈账本

调用前 rsp=0x1000，执行 call 后 rsp=0x0ff8，该处为返回地址；push rbp 后 rsp=0x0ff0；mov rsp,rbp 令 rbp=0x0ff0；sub $32,rsp 后 rsp=0x0fd0。若局部值放-8(rbp)，地址为 0x0fe8。此时局部空间、旧 rbp、返回地址占不同位置。

执行 leave 相当于恢复 rsp 到 rbp 再弹出旧 rbp，rsp 回 0x0ff8；ret 弹返回地址后回 0x1000。逐行写栈顶变化，就不会把局部变量和返回地址混在同一格。

18.6 题六：32 位调用的另一套偏移

传统 cdecl 调用 f(3,4)时，先压 4 再压 3，再 call；函数入口[esp]是返回地址，[esp+4]是 3，[esp+8]是 4。若接着 push ebp、mov esp,ebp，则[ebp+8]是第一个参数。这不是 x86-64 的“前六个整数参数用寄存器”，题目环境必须先读。

18.7 题七：结构体重排

在 long 对齐 8、short 对齐 2 的常见 LP64 ABI 中，`char,long,short` 布局偏移可为 0、8、16，总大小 24；改成 `long,short,char`，偏移 0、8、10，总大小 16。省空间来自减少填充，不是缩短某个 long。若布局用于文件格式、ABI 或外部协议，不能为了省空间擅自改字段顺序。

18.8 题八：从条件码看有符号小于

八位 127 减(-1)，数学结果 128，机器低位 0x80 为负，SF=1；该减法有符号溢出，OF=1。若只看 SF 就误以为 127<−1；真正 jl 检查 SF≠OF，此时为假，得到正确比较结论。这个例子说明为什么必须把 OF 带上。

练习答案检查法。写出原值、原位宽、目标位宽、位串、解释规则五列；地址题另加“基址/步长/下标/字段偏移”；栈题每条指令一行。不要把关键中间步骤省掉，然后只检查最后一个十六进制数。

19 系统综合题：把“偶尔错”变成可解释的事件顺序

19.1 题一：fork 后的文件偏移

父进程 open 一个普通文件，再 fork；父子继承的对应 fd 指向同一个打开文件描述，因此文件偏移可能共享。若双方各 read 一个字节，通常读取的是共享偏移处相继的字节，谁先获得哪一个不确定。若父子在 fork 之后各自独立 open，通常就各有独立偏移，可能都读到首字节。先画“进程 fd 表→打开文件描述→文件”的箭头，再回答，不能只数 open 调用总数。

19.2 题二：子进程比父进程先退出

顺序可能是 fork 成功→子 exit→SIGCHLD 递送→处理函数试删任务表→父才把 PID 加入。修复用阻塞 SIGCHLD 把 fork 与添加表项包成相对于信号递送的受保护区；子分支恢复掩码。处理函数循环回收，因为多个退出可能合并成一个标准信号。

这里涉及两种不同问题：表项操作顺序的竞态，以及通知次数不可当精确退出计数。只修其中一项仍可能留下错误。

19.3 题三：页表与 TLB

4 KiB 页、虚拟地址 0x12345。VPN=0x12，偏移 0x345。TLB miss 但页表有效且页驻留，硬件/内核按体系结构取到映射后填 TLB，正常继续；若页表说明合法但不驻留，触发缺页调入；若根本不合法，则不是“随便找页框装入”就能修复。题目给的 valid 位含义必须按其页表格式读，有的教学表还把“合法”“驻留”拆成不同字段。

19.4 题四：malloc 失败后如何保住原数据

原 p 指向有效块，想扩到 new_size。先用临时 q=realloc(p,new_size)。q 为空时保留 p 并处理失败；q 非空才令 p=q。若直接 p=realloc(...)，失败会丢掉唯一指针，造成泄漏。成功且移动后，旧 p 与旧元素引用不能继续使用。

19.5 题五：读足一个报文

报文头说长度 100，第一次 read 得 40，第二次得 35，第三次得 25。正确循环依次写 buffer+0、+40、+75，剩余 100、60、25。若每次都写 buffer 起点，会覆盖前面数据；若每次仍请求 100，还可能越过缓冲剩余空间。若第三次返回 0，则只得到 75，必须报告提前结束，不得把未初始化尾部当已收到数据。

19.6 题六：生产者消费者的死锁

缓冲满，生产者先拿 mutex 再 P(empty)而睡；消费者想拿 mutex 才能取走物品。生产者等消费者腾空，消费者等生产者释放锁，形成环。正确顺序先等资源计数，再进临界区；但也要保证出错/取消路径不会拿走计数却不归还。

19.7 题七：代码正确却慢

同样求矩阵所有元素和，行序与列序大 O 都相同，但步长不同引发缓存/TLB 差别。先测量总时间与访问规模，检查是否含初始化/打印，再讨论局部性；不要拿一次包含磁盘读入的耗时声称 CPU 循环优化快了十倍。

19.8 独立复述题

用五句话分别解释：signed 溢出与机器回绕；动态库与静态库；进程与线程；TLB miss 与缺页；条件变量与计数信号量。每一组至少举一个“看着像、实际不同”的具体例子，能说清例子，比只背术语定义更接近掌握。

资料覆盖、原图出处与使用说明

这是一份独立组织、重新讲解的教材式初稿。课程范围首先按教师资料；并非教师审核的官方教材，也不把补充证明和预习部分自动列作考试要求。

章节覆盖表

教学来源	本书对应
第 01 讲课程引言	00、08—17、19：程序全景及后续课程范围
第 02 讲整数	01—02、18：位宽、端序、转换与溢出
第 03 讲浮点（含修订页）	03、18：正规/非正规、特殊值、舍入
第 04 讲机器表示 1	04、07：指令、寻址与布局
第 05 讲机器表示 2	05、18：条件码、循环、switch
第 06 讲机器表示 3	06、18：调用、递归、32/64 位差异
HW1、HW2 题面与 AttackLab 说明	02—07、18—19 的通用方法；不收录私人提交
引言列出的 CSAPP 2/3/7/8/9/10/12 章选节	对应本册主干；分配器按 9.9.12 之前的引导范围解释，多级页表不作默认要求

原生解析与视觉核对

PDF 提取原生文字与页码，PPT/PPTX 提取文字和内嵌图片，同时原生兼容导出完整幻灯片以保留箭头、布局、曲线与公式。机器已为可用课件生成完整页级联系表；写入笔记的关键原图逐幅视觉核对。未声称已对数千页的每一个细节完成逐字人工审校。PPT 导出中缺失中文字体替换为 Noto Sans CJK SC，原件不覆盖。

原生提取和视觉复核需要互相补足：抽出的字不能告诉你箭头指谁，图片也不总能准确给出公式符号。课件中的省略或不一致处，本书在相关段落说明，不把它们无条件当作定理。

原图逐项出处

- 老师原图-整数.png：计算机系统概论/slides/2.1-整数.pdf，PDF 第 7 页。直接渲染原始 PDF 页，不改图中结构。
- 老师原图-浮点.png：计算机系统概论/slides/2.2-浮点数.pdf，PDF 第 13 页。直接渲染原始 PDF 页，不改图中结构。
- 老师原图-调用栈.png：计算机系统概论/slides/第 06 讲-C 语言的机器表示-3.pdf，PDF 第 4 页。直接渲染原始 PDF 页，不改图中结构。
- 老师原图-栈帧布局.png：计算机系统概论/slides/第 06 讲-C 语言的机器表示-3.pdf，PDF 第 19 页。直接渲染原始 PDF 页，不改图中结构。
- 老师原图-switch.png：计算机系统概论/slides/第 05 讲-C 语言的机器表示-2.pdf，PDF 第 43 页。直接渲染原始 PDF 页，不改图中结构。

原创 SVG 图由工具脚本/教材/配图.py 生成，文字转为轮廓，可离线显示；它们是本书的补充示意，不伪装成教师原图。

版本和复核边界

2026-10-25，用新版 OneTHU 默认共享后台只读核验课程文件清单，三门课文件枚举均返回 complete=true。数据结构的综合资源接口对作业全量枚举仍给出未核实状态，因此本书不宣称覆盖了独立 OJ 未来/全部题目。随后已从作者官方入口取得 669 页主教材与《习题解析》合体 PDF；数据结构 32—33 章补充 1-32 的 16 种程序推导和全部 12 章交叉索引。取得全文不等于已经把其中 292 道大题所有小问逐一重做，具体覆盖按各章说明。

完整文字、公式与图在 Markdown 和 Typst 间共享；每次改正文后应运行构建器，Pandoc 公式转换警告将阻止成功构建。重要例题按推导、量纲、边界和独立小规模程序交叉检查，发现错误应同时修正文并重建两版本。

延伸阅读：CSAPP 官方配套页。本书以本地教师课件为直接范围来源，延伸链接供查原始定义与工具。